

CADASTRO DE USUÁRIO

NOME COMPLETO

Digite seu nome

IDADE

Digite sua idade

CADASTRAR

php

STATUS DO PROCESSAMENTO

USUÁRIO CADASTRADO

Nome: Flávio De Lira
Idade: 35 anos
Status: [ACESSO CONCEDIDO] - Credencial Ativa

USUÁRIO CADASTRADO

Nome: Arlete De Lira
Idade: 25 anos
Status: [ACESSO CONCEDIDO] - Credencial Ativa

USUÁRIO CADASTRADO

Nome: Looney Tunes
Idade: 2 anos
Status: [RESTRITO] - Aguardando Autorização do Guardião

[VOLTAR AO TERMINAL]

Funções e Estruturas de Repetição

Função Personalizada

- validarCadastro(string \$nome, int \$idade): array:** Recebe o nome e a idade do usuário, higieniza o texto, verifica a maioridade e retorna um array associativo com os dados formatados.

Funções Nativas do PHP

- trim(\$nome):** Remove espaços extras no início e no final do nome.
- strtolower(...):** Converte todo o nome para letras minúsculas.
- ucwords(...):** Converte a primeira letra de cada palavra do nome para maiúscula.
- isset(\$_POST['...']):** Verifica se a chave botaoCadastrar existe no envio da requisição POST.
- empty(...):** Valida se o array de cadastros processados possui elementos antes da exibição.

foreach (\$cadastros as \$pessoa) (Processamento Back-end): Itera sobre o array multidimensional \$cadastros. Em cada ciclo, passa os dados de \$pessoa para a função validarCadastro() e insere o retorno no array \$listaUsuarios[].

<?php foreach (\$listaUsuarios as \$usuario): ?> ... <?php endforeach; ?> (Renderização Front-end): Utiliza a sintaxe alternativa do PHP para templates HTML, percorrendo \$listaUsuarios para gerar dinamicamente os blocos visuais (.alert-box) de cada cadastro.

Principais decisões de desenvolvimento



Condicional Ternário

Simplifica a checagem de idade (\$idade >= 18) em uma única linha, sem usar if/else verboso.

Encapsulamento

A função validarCadastro() isola a validação e formatação, evitando repetição de código.

Processamento em Lote

Junta o dado do formulário com registros fixos para testar múltiplos cenários de uma vez.

Sintaxe Alternativa no HTML

Uso de if: e foreach: com endif; deixa o código limpo ao misturar PHP e HTML.

Separação Lógica/HTML

O PHP processa os dados no início e o HTML apenas os exibe no final.

Desafio Intercalação de Código Server-Side no Front-End

O Problema

- Confusão ao abrir e fechar tags do PHP (<?php ... ?>) no meio do HTML, com risco de chaves { } soltas e erros ao tentar exibir dados vazios.

A Solução

Sintaxe Alternativa

Troca das chaves por if: / endif; e foreach: / endforeach; deixando o HTML mais limpo.

Blocos Dinâmicos:

Colocação do foreach apenas na div .alert-box, criando um card por usuário de forma automática.

Trava de Segurança:

Uso do teste if (\$enviado && !empty(\$listaUsuarios)) para só gerar os cards se o formulário for enviado com dados.

Aprendizados do processo



Isolamento e Reutilização de Código

Criar funções personalizadas para validação evita repetições e centraliza regras de negócio, facilitando a manutenção.

Sanitização de Dados

O tratamento de dados no back-end com funções de manipulação de strings garante padronização das informações recebidas.

Controle de Fluxo Limpo no Front-End

A sintaxe alternativa do PHP (if: / endif;, foreach: / endforeach;) resolve a complexidade de misturar lógica e HTML, mantendo a marcação clara.

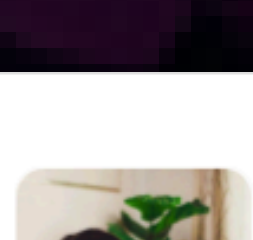
Validação Condicional Prévia

Verificar se os dados existem e foram submetidos antes de rodar laços de repetição previne erros de variáveis indefinidas e renderizações indesejadas.

Processamento Dinâmico em Lote

O uso de estruturas de repetição permite gerar componentes visuais dinamicamente a partir de arrays de dados.

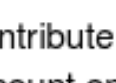
arletedelira-TI/etec-php-agenda04-ds-ii



1 Contribuidor 0 Issues 0 Stars 0 Forks

arletedelira-TI/etec-php-agenda04-ds-ii

Contribute to arletedelira-TI/etec-php-agenda04-ds-ii development by creating an account on GitHub.



Acesse o Repositório

